

HUBS — one row per unique business key, inserted the first time the key is seen

HUB_CUSTOMER	
CUSTOMER_HK (PK)	BINARY(16)
CUSTOMER_NUMBER	VARCHAR(20)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

HUB_ORDER	
ORDER_HK (PK)	BINARY(16)
ORDER_NUMBER	VARCHAR(20)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

HUB_PRODUCT	
PRODUCT_HK (PK)	BINARY(16)
PRODUCT_CODE	VARCHAR(30)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

HUB_CHANNEL	
CHANNEL_HK (PK)	BINARY(16)
CHANNEL_CODE	VARCHAR(20)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

HUB_PAYMENT_METHOD	
PAYMENT_METHOD_HK (PK)	BINARY(16)
PAYMENT_CODE	VARCHAR(20)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

HUB_SUPPLIER	
SUPPLIER_HK (PK)	BINARY(16)
SUPPLIER_CODE	VARCHAR(20)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

LINKS — one row per unique combination of Hub keys (the “unit of work”)

LNK_ORDER_ITEM	
ORDER_ITEM_HK (PK)	BINARY(16)
ORDER_HK (FK)	BINARY(16)
PRODUCT_HK (FK)	BINARY(16)
CUSTOMER_HK (FK)	BINARY(16)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

LNK_ORDER_CHANNEL	
ORDER_CHANNEL_HK (PK)	BINARY(16)
ORDER_HK (FK)	BINARY(16)
CHANNEL_HK (FK)	BINARY(16)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

LNK_ORDER_PAYMENT	
ORDER_PAYMENT_HK (PK)	BINARY(16)
ORDER_HK (FK)	BINARY(16)
PAYMENT_METHOD_HK (FK)	BINARY(16)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

LNK_PRODUCT_SUPPLY	
PRODUCT_SUPPLY_HK (PK)	BINARY(16)
PRODUCT_HK (FK)	BINARY(16)
SUPPLIER_HK (FK)	BINARY(16)
LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

WHAT IS A HUB?

The unique, deduplicated list of ONE business key (customer number, order number...). Inserted once, never updated. Its hash key (_HK) is the anchor every Link and Satellite points to.

WHAT IS A LINK?

Records that two or more Hub keys occurred together: an order line, a payment, a supply arrangement. It holds ONLY hash keys + audit columns — no amounts, and no begin/end dates (those live in Satellites → Page 2).

WHY HASH KEYS?

_HK = MD5 / SHA-256 of the business key(s), e.g. CUSTOMER_HK = HASH(CUSTOMER_NUMBER). Deterministic, so loaders run in parallel with no lookups or sequence generators; every join is one BINARY(16) column.

AUDIT COLUMNS ON EVERY ROW

LOAD_DTS — when the warehouse first saw this row; REC_SRC — which source system delivered it. Together with insert-only loading they give a complete, replayable audit trail of everything ever received.

READ THE MODEL IN COLOUR

Green = Hub (keys) Purple = Link (relationships) Amber = Satellite (history)
Teal = Point-in-Time Blue = Bridge Crimson = Reporting Slate = Staging
Pages 1–2 Raw Vault; Page 3 Info Mart; Page 5 diagram; Pages 6–9 example, SQL & PIT/Bridge; 10–12 Genesys source mapping.

HUB SATELLITES — attached to a Hub; split by rate of change (profile vs address, details vs stock)

SAT_CUSTOMER_PROFILE	
CUSTOMER_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
FULL_NAME	NVARCHAR(100)
EMAIL_ADDRESS	NVARCHAR(200)
PHONE_NUMBER	VARCHAR(20)
LOYALTY_TIER	VARCHAR(20)

SAT_CUSTOMER_ADDRESS	
CUSTOMER_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
STREET_ADDRESS	NVARCHAR(200)
SUBURB	NVARCHAR(100)
STATE_CODE	CHAR(3)
POSTCODE	CHAR(10)

SAT_PRODUCT_DETAILS	
PRODUCT_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
PRODUCT_NAME	NVARCHAR(200)
CATEGORY	NVARCHAR(100)
BRAND	NVARCHAR(100)
UNIT_PRICE	DECIMAL(10,2)

SAT_PRODUCT_STOCK	
PRODUCT_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
STOCK_ON_HAND	INT
REORDER_LEVEL	INT
WAREHOUSE_ZONE	VARCHAR(10)

LINK SATELLITES — attached to a Link; descriptive measures, status history and effectivity

SAT_ORDER_ITEM_DETAILS	
ORDER_ITEM_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
QUANTITY	INT
UNIT_PRICE_SOLD	DECIMAL(10,2)
DISCOUNT_PCT	DECIMAL(5,2)
LINE_TOTAL	DECIMAL(12,2)

SAT_ORDER_STATUS	
ORDER_ITEM_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
ORDER_STATUS	VARCHAR(30)
DELIVERY_STATUS	VARCHAR(30)
TRACKING_NUMBER	VARCHAR(50)
EST_DELIVERY_DATE	DATE

SAT_PAYMENT_DETAILS	
ORDER_PAYMENT_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
HASH_DIFF	BINARY(16)
REC_SRC	VARCHAR(50)
PAYMENT_AMOUNT	DECIMAL(12,2)
PAYMENT_STATUS	VARCHAR(20)
PAYMENT_DATE	DATETIME2
GATEWAY_REF	VARCHAR(100)

SAT_ORDER_EFFECTIVITY	
ORDER_ITEM_HK (PK, FK)	BINARY(16)
LOAD_DTS (PK)	DATETIME2
REC_SRC	VARCHAR(50)
EFFECTIVE_FROM	DATETIME2
EFFECTIVE_TO	DATETIME2
IS_DELETED	BIT

COMPOSITE PRIMARY KEY

PK = parent_HK + LOAD_DTS, so every historical version of the same Customer / order line is a unique, queryable row.

HASH_DIFF — CHANGE DETECTION

Hash of all descriptive columns. Compare staged HASH_DIFF to the latest stored one: different → insert new row; same → skip.

SPLIT BY RATE OF CHANGE

Hot attributes (stock, status) and cold ones (name, brand) live in separate Satellites, so fast change doesn't bloat slow history.

EFFECTIVITY SATELLITE

Links carry no dates, so validity lives here: EFFECTIVE_FROM / TO, IS_DELETED track when an order line applies or is cancelled.

INSERT-ONLY — THE CORE RAW VAULT RULE

No row in any Hub, Link or Satellite is ever physically updated or deleted. Corrections, status changes and re-deliveries all arrive as new rows with a later LOAD_DTS. That makes loads idempotent and re-runnable, keeps a full audit trail, and lets any past state be reconstructed — see the worked example on Page 6.

POINT-IN-TIME (PIT) TABLES — snapshot of the latest Satellite LOAD_DTS per parent key

PIT_CUSTOMER	
SNAPSHOT_DTS (PK)	DATETIME2
CUSTOMER_HK (PK, FK)	BINARY(16)
SAT_PROFILE_LOAD_DTS	DATETIME2
SAT_ADDRESS_LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

PIT_ORDER_ITEM	
SNAPSHOT_DTS (PK)	DATETIME2
ORDER_ITEM_HK (PK, FK)	BINARY(16)
SAT_ITEM_LOAD_DTS	DATETIME2
SAT_STATUS_LOAD_DTS	DATETIME2
SAT_EFFECT_LOAD_DTS	DATETIME2
REC_SRC	VARCHAR(50)

HOW PIT TABLES WORK

At regular intervals (hourly / daily) a PIT row captures the most recent LOAD_DTS of every Satellite for a given Hub or Link key. Without PIT: each Satellite joins via slow date-range (BETWEEN) predicates. With PIT: one equal-join on SNAPSHOT_DTS returns all correct Satellite versions at once.

BRIDGE TABLE — pre-joins Hub and Link keys into one snapshot: a base-level fact-table skeleton

BRDG_ORDER_SALES	
SNAPSHOT_DTS (PK)	DATETIME2
ORDER_ITEM_HK (PK, FK)	BINARY(16)
CUSTOMER_HK (FK)	BINARY(16)
ORDER_HK (FK)	BINARY(16)
PRODUCT_HK (FK)	BINARY(16)
CHANNEL_HK (FK)	BINARY(16)
PAYMENT_METHOD_HK (FK)	BINARY(16)

HOW BRIDGE TABLES WORK

Combines the keys of multiple Hubs and Links into a single pre-joined snapshot at the grain of the order line. Analysts query BRDG_ORDER_SALES instead of walking Hub → Link → Satellite chains: the Bridge already knows which customer ordered which product via which channel.

DATA FLOW TO THE BUSINESS

Raw Vault (Pages 1–2)
↓
PIT / Bridge tables
↓
soft business rules applied
↓
Reporting tables → Power BI / Tableau

REPORTING TABLE — flat, denormalised, soft business rules applied; what Power BI / Tableau query

RPT_SALES_FACT (columns 1–8)	
ORDER_ITEM_KEY (PK)	BIGINT
ORDER_DATE	DATE
CUSTOMER_NUMBER	VARCHAR(20)
CUSTOMER_NAME	NVARCHAR(100)
PRODUCT_CODE	VARCHAR(30)
PRODUCT_NAME	NVARCHAR(200)
CATEGORY	NVARCHAR(100)
CHANNEL_CODE	VARCHAR(20)

RPT_SALES_FACT (columns 9–15)	
QUANTITY	INT
UNIT_PRICE_SOLD	DECIMAL(10,2)
DISCOUNT_PCT	DECIMAL(5,2)
LINE_TOTAL	DECIMAL(12,2)
ORDER_STATUS	VARCHAR(30)
PAYMENT_STATUS	VARCHAR(20)
DW_LOAD_DTS	DATETIME2

DISPOSABLE BY DESIGN

PIT, Bridge and Reporting tables are derived, system-driven structures: they can be dropped and rebuilt from the Raw Vault at any time without losing a single fact. Business keys (CUSTOMER_NUMBER...) are resolved back from the Hubs; names and amounts come from the Satellites.



ENTITY PREFIX REFERENCE

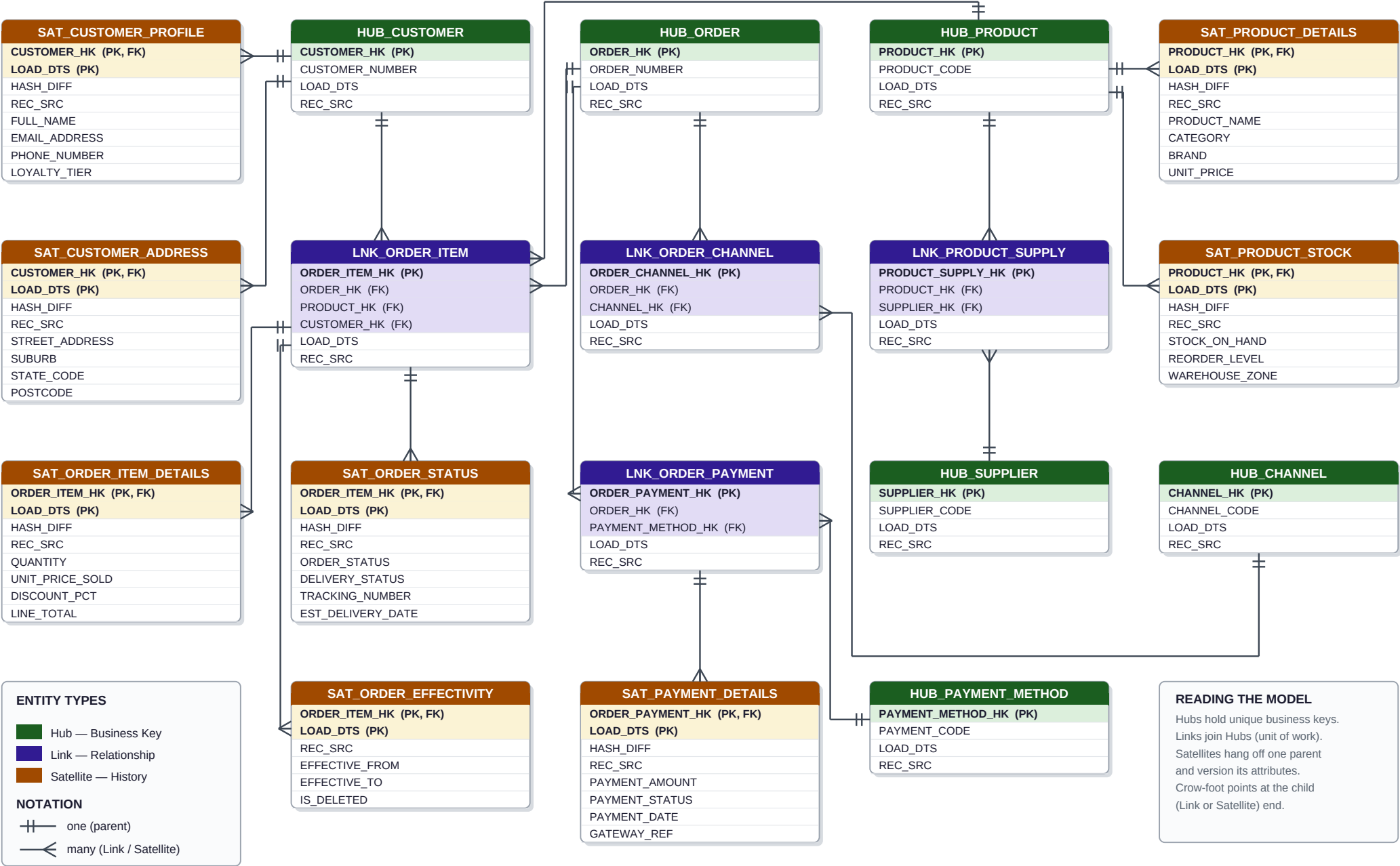
HUB_	Hub — unique list of business keys
LNK_	Link — relationships between Hubs
SAT_	Satellite — descriptive data over time
SAL_	Same-As Link — synonym key mapping
HLNK_	Hierarchical Link — parent-child
STG_	Staging table
PIT_	Point-in-Time table
BRDG_	Bridge table
BHUB_ / BLNK_ / BSAT_	Business Vault Hub / Link / Satellite
RPT_	Reporting table (Information Mart)
VDIM_ / VF_	Info-Mart dimension / fact views

FIELD NAMING (SUFFIX) REFERENCE

_HK	hash key (BINARY 16) — e.g. CUSTOMER_HK
LOAD_DTS	warehouse load timestamp
REC_SRC	source system of the row
HASH_DIFF	change-detection hash (Satellites)
SNAPSHOT_DTS	PIT / Bridge snapshot timestamp

KEY DATA VAULT 2.0 RULES

- INSERT-ONLY: nothing in the Raw Vault is ever physically updated or deleted after insertion.
- HASH KEYS, not sequences: _HK = MD5 / SHA-256 of the business key(s).
- NO DATES ON LINKS: validity belongs in Effectivity Satellites (SAT_ORDER_EFFECTIVITY).
- SPLIT SATELLITES by rate of change.
- HARD rules (typing, hashing) before the vault; SOFT rules (business logic) only after it.



1 SOURCE EVENTS arrive from the Order Management System / CRM

STAGING — raw business events		
EVENT	TIMESTAMP	WHAT HAPPENED
T1	2026-06-10 09:15	Order ORD-10001 — customer CUST-501 (Priya Sharma, priya.s@mail.com) buys 2 × PRD-2001 Wireless Mouse @ \$29.95 via WEB, pays CARD \$59.90
T2	2026-06-10 11:42	Order ORD-10002 — customer CUST-502 (Liam Chen, liam.c@mail.com) buys 1 × PRD-2002 Mechanical Keyboard @ \$89.00 via APP, pays PAYPAL \$89.00
T3	2026-06-11 08:05	CRM change — CUST-501 updates email to priya@newmail.com (no new order, no new customer)
T4	2026-06-11 16:20	Fulfilment update — ORD-10001 line status changes PLACED → SHIPPED

2 HUBS get one row per NEW business key only — T3 and T4 add nothing here (keys already exist)

HUB_CUSTOMER			
CUSTOMER_HK (PK)	CUSTOMER_NUMBER	LOAD_DTS	REC_SRC
9aF2...	CUST-501	06-10 09:15	OMS
4bC7...	CUST-502	06-10 11:42	OMS

HUB_ORDER			
ORDER_HK (PK)	ORDER_NUMBER	LOAD_DTS	REC_SRC
7e1D...	ORD-10001	06-10 09:15	OMS
2cA9...	ORD-10002	06-10 11:42	OMS

HUB_PRODUCT			
PRODUCT_HK (PK)	PRODUCT_CODE	LOAD_DTS	REC_SRC
5fB3...	PRD-2001	06-10 09:15	OMS
8dE6...	PRD-2002	06-10 11:42	OMS

3 LINKS record the relationship — one row per unique combination of hub keys (the order line)

LNK_ORDER_ITEM					
ORDER_ITEM_HK (PK)	ORDER_HK	PRODUCT_HK	CUSTOMER_HK	LOAD_DTS	REC_SRC
L1a4...	7e1D...	5fB3...	9aF2...	06-10 09:15	OMS
L9c2...	2cA9...	8dE6...	4bC7...	06-10 11:42	OMS

How the Link row is built

L1a4... = HASH(ORD-10001 + PRD-2001 + CUST-501).
It carries NO quantities, prices or dates of business meaning — only keys. T1 created row 1, T2 created row 2. T3 / T4 change nothing: same combinations.

4 SATELLITES hold the descriptive detail and its history — a change = a NEW row (insert-only), never an update

SAT_CUSTOMER_PROFILE (parent: HUB_CUSTOMER)					
CUSTOMER_HK (PK,FK)	LOAD_DTS (PK)	HASH_DIFF	REC_SRC	FULL_NAME	EMAIL_ADDRESS
9aF2...	06-10 09:15	aa31...	OMS	Priya Sharma	priya.s@mail.com
4bC7...	06-10 11:42	b7c2...	OMS	Liam Chen	liam.c@mail.com
9aF2...	06-11 08:05	e09f...	CRM	Priya Sharma	priya@newmail.com

SAT_ORDER_ITEM_DETAILS (parent: LNK_ORDER_ITEM)						
ORDER_ITEM_HK (PK,FK)	LOAD_DTS (PK)	HASH_DIFF	REC_SRC	QTY	UNIT_PRICE	LINE_TOTAL
L1a4...	06-10 09:15	91ab...	OMS	2	29.95	59.90
L9c2...	06-10 11:42	c4d0...	OMS	1	89.00	89.00

SAT_ORDER_STATUS (parent: LNK_ORDER_ITEM)					
ORDER_ITEM_HK (PK,FK)	LOAD_DTS (PK)	HASH_DIFF	REC_SRC	ORDER_STATUS	DELIVERY_STATUS
L1a4...	06-10 09:15	5e1b...	OMS	PLACED	PENDING
L9c2...	06-10 11:42	5e1b...	OMS	PLACED	PENDING
L1a4...	06-11 16:20	9c4f...	ERP	SHIPPED	IN_TRANSIT

WHY EVERY SATELLITE CARRIES HASH_DIFF + REC_SRC (Page 2 standard)

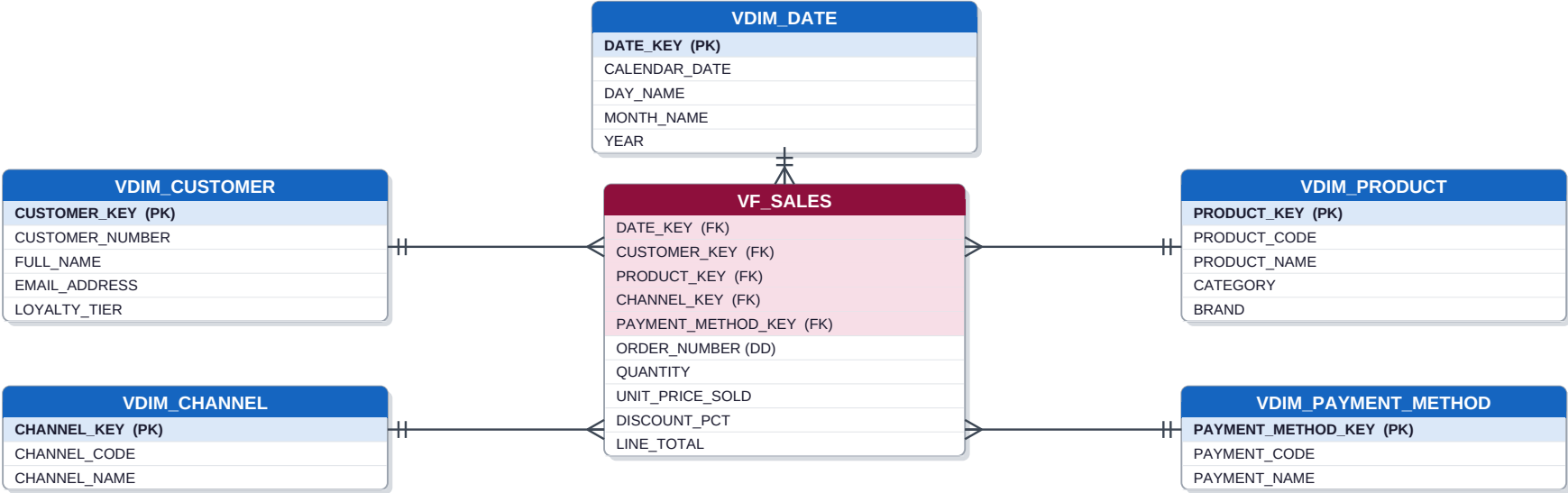
HASH_DIFF = hash of the descriptive columns. T3's row hashes to e09f... ≠ stored aa31... → insert; replaying T1 would hash to aa31... again → skip. Identical attributes give identical hashes (both PLACED rows = 5e1b...) — that's fine: uniqueness comes from the PK. REC_SRC shows who delivered each row: OMS orders, CRM change, ERP shipping.

 = new version row written by T3 / T4 — the earlier row stays untouched, giving full history. Satellite PK = parent_HK + LOAD_DTS (Page 2).

5 WHAT THIS SHOWS — AND WHERE IT GOES NEXT

Hubs answer “who / what exists?” · Links answer “what relates to what?” · Satellites answer “what was true, when?”
CUST-501 exists once forever; one Link row per order line; Priya’s old and new email are both queryable by LOAD_DTS. Page 7 reshapes exactly these rows into a star schema for Power BI / Tableau, and Page 8 shows the PostgreSQL that loads the vault and publishes that star.

STAR SCHEMA — grain of VF_SALES: one row per order line (= one LNK_ORDER_ITEM key)



POPULATED WITH THE PAGE 6 EXAMPLE — the current (Type 1) view

VF_SALES									
DATE_KEY	CUSTOMER_KEY	PRODUCT_KEY	CHANNEL_KEY	PAYMENT_METHOD_KEY	ORDER_NUMBER	QTY	UNIT_PRICE	DISCOUNT_PCT	LINE_TOTAL
20260610	9aF2...	5fB3...	6cD1...	7aB5...	ORD-10001	2	29.95	0.00	59.90
20260610	4bC7...	8dE6...	3eF8...	1dC4...	ORD-10002	1	89.00	0.00	89.00

THE 2 FACT ROWS

One row per order line from Page 6:
ORD-10001 (Priya) and ORD-10002 (Liam).

VDIM_CUSTOMER				
CUSTOMER_KEY	CUSTOMER_NUMBER	FULL_NAME	EMAIL_ADDRESS	LOYALTY_TIER
9aF2...	CUST-501	Priya Sharma	priya@newmail.com	GOLD
4bC7...	CUST-502	Liam Chen	liam.c@mail.com	SILVER

VDIM_PRODUCT				
PRODUCT_KEY	PRODUCT_CODE	PRODUCT_NAME	CATEGORY	BRAND
5fB3...	PRD-2001	Wireless Mouse	Accessories	LogiPro
8dE6...	PRD-2002	Mechanical Keyboard	Accessories	KeyCo

VDIM_DATE		
DATE_KEY	CALENDAR_DATE	DAY_NAME
20260610	2026-06-10	Wednesday
20260611	2026-06-11	Thursday

= value delivered by T3 — a Type 1 dimension always carries the LATEST satellite version (Priya's new email), while the vault still holds the old one.

HOW EACH COLUMN IS SOURCED

Keys come straight from the vault: CUSTOMER_KEY
= CUSTOMER_HK (9aF2...); DATE_KEY from LOAD_DTS.
Measures = latest SAT_ORDER_ITEM_DETAILS row;
ORDER_NUMBER from HUB_ORDER stays on the fact
as a degenerate dimension (DD).

TYPE 1 NOW, TYPE 2 WHENEVER NEEDED

Dimensions show current values only (Type 1) —
Priya's new email. The vault keeps every version
(Page 6), so VDIM_CUSTOMER can be rebuilt as a
Type 2 SCD with validity dates at any time, with
no re-sourcing from operational systems.

WHY BI ENGINES LOVE THIS SHAPE

Single-column joins, additive fact measures,
small dimensions: what VertiPaq (Power BI) and
Tableau extracts optimise for. Import VF_SALES +
VDIM_ views; model one-to-many relationships
from each dimension to the fact.

FROM VAULT TO STAR

These are views (or materialised tables) in the Information Mart layer of Page 4, built by the PostgreSQL on Page 8. The Raw Vault remains the single
source of truth — the star is disposable and can be reshaped or rebuilt at will.

1 · LOAD HUBS — insert business keys never seen before

```

INSERT INTO hub_customer
(customer_hk, customer_number, load_dts, rec_src)
SELECT DISTINCT
    md5(upper(trim(s.customer_number))::uuid, -- one hash recipe,
    s.customer_number, -- used everywhere
    s.load_dts,
    s.rec_src
FROM
    stg_orders s
WHERE
    NOT EXISTS (
        SELECT 1 FROM hub_customer h
        WHERE
            h.customer_hk =
                md5(upper(trim(s.customer_number))::uuid );

-- Page 6: T1 inserts CUST-501, T2 inserts CUST-502.
-- T3 / T4 insert nothing: the keys already exist.

```

3 · LOAD SATELLITES — new row only when HASH_DIFF changes

```

WITH src AS (
    SELECT md5(upper(trim(customer_number))::uuid AS customer_hk,
        load_dts,
        md5(concat_ws('|', full_name, email_address,
            phone_number, loyalty_tier))::uuid AS hash_diff,
        rec_src,
        full_name, email_address, phone_number, loyalty_tier
    FROM
        stg_customers )
INSERT INTO sat_customer_profile
SELECT s.*
FROM
    src s
LEFT JOIN LATERAL (
    SELECT p.hash_diff
    FROM
        sat_customer_profile p
    WHERE
        p.customer_hk = s.customer_hk
    ORDER
        BY p.load_dts DESC
    LIMIT 1 ) cur ON true
WHERE
    cur.hash_diff IS DISTINCT FROM s.hash_diff;

-- T3: Priya's email changed -> new HASH_DIFF -> one new row.
-- Replaying T1 tomorrow: identical hash -> 0 rows. Idempotent.

```

2 · LOAD LINKS — insert key combinations never seen before

```

INSERT INTO lnk_order_item
(order_item_hk, order_hk, product_hk, customer_hk,
load_dts, rec_src)
SELECT DISTINCT
    md5(concat_ws('|', s.order_number, s.product_code,
        s.customer_number))::uuid,
    md5(s.order_number)::uuid,
    md5(s.product_code)::uuid,
    md5(upper(trim(s.customer_number))::uuid,
    s.load_dts, s.rec_src
FROM
    stg_orders s
WHERE
    NOT EXISTS (
        SELECT 1 FROM lnk_order_item l
        WHERE
            l.order_item_hk = md5(concat_ws('|',
                s.order_number, s.product_code,
                s.customer_number))::uuid );

```

4 · PUBLISH THE STAR — the views Power BI / Tableau import (Page 7)

```

CREATE OR REPLACE VIEW vdim_customer AS
SELECT h.customer_hk AS customer_key,
    h.customer_number, p.full_name,
    p.email_address, p.loyalty_tier
FROM
    hub_customer h
JOIN
    LATERAL (
        SELECT * FROM sat_customer_profile sp
        WHERE
            sp.customer_hk = h.customer_hk
        ORDER
            BY sp.load_dts DESC LIMIT 1 ) p ON true;

CREATE OR REPLACE VIEW vf_sales AS
SELECT to_char(l.load_dts,'YYYYMMDD')::int AS date_key,
    l.customer_hk AS customer_key,
    l.product_hk AS product_key,
    o.order_number, -- degenerate dimension
    d.quantity, d.unit_price_sold,
    d.discount_pct, d.line_total
FROM
    lnk_order_item l
JOIN
    hub_order o USING (order_hk)
JOIN
    LATERAL (
        SELECT * FROM sat_order_item_details sd
        WHERE
            sd.order_item_hk = l.order_item_hk
        ORDER
            BY sd.load_dts DESC LIMIT 1 ) d ON true;

```

RUNNING IT IN PRODUCTION

- Orchestration order (Airflow / dbt / cron): stage → all Hubs in parallel → all Links in parallel → all Satellites in parallel → PIT / Bridge → star views.
- Idempotent by construction: hash keys + NOT EXISTS + HASH_DIFF mean a failed batch is simply re-run — no duplicates, no clean-up scripts (the Raw Vault rule from Page 2).
- At scale: materialise VF_SALES / VDIM_* as tables or incremental dbt models, and replace the LATERAL-latest join with the PIT equal-join from Page 3.
- Same pattern, other engines: SQL Server uses HASHBYTES('SHA2_256', ...) into BINARY columns; Snowflake / Databricks use md5() / sha2() — only the hash function changes.
- Hard rules (typing, trimming, hashing) happen in staging, before the vault; soft business rules live only in the Business Vault / Information Mart (Page 4).

The same two orders + email change + status change from Page 6, now snapshotted into a Point-in-Time and a Bridge table for join-free querying

Information Mart · system-driven · rebuilt from the Raw Vault

THE PROBLEM PIT & BRIDGE SOLVE

From Page 6, CUST-501’s profile has TWO versions (old + new email) and her order line has TWO status versions (PLACED, then SHIPPED). To report “what was true on 11 June” you must pick the right LOAD_DTS in EACH satellite with date-range logic. A PIT pre-picks those dates; a Bridge pre-joins the keys. Both are just snapshots.

RECAP — the satellite versions from Page 6 that a 2026-06-11 snapshot must resolve

▲ CHANGED — email updated at 06-11 08:05			✓ UNCHANGED — address never changed — one row only			▲ CHANGED — status changed to SHIPPED at 16:20			“AS-OF” IS THE HARD PART		
SAT_CUSTOMER_PROFILE (2 versions — CUST-501 = 9aF2...)			SAT_CUSTOMER_ADDRESS (1 version — CUST-501 = 9aF2...)			SAT_ORDER_STATUS (2 versions — line L1a4...)			Snapshot at 2026-06-11 23:59 must choose the LATEST LOAD_DTS ≤ that instant in EACH satellite: PROFILE → 08:05 (email changed) ADDRESS → 06-10 09:15 (never changed; stays on its original row) STATUS → 16:20 (SHIPPED) CUSTOMER_HK 9aF2... is identical in all three — only the per-satellite LOAD_DTS differs.		
CUSTOMER_HK	LOAD_DTS	EMAIL_ADDRESS	CUSTOMER_HK	LOAD_DTS	SUBURB	ORDER_ITEM_HK	LOAD_DTS	ORDER_STATUS			
9aF2...	2026-06-10 09:15	priya.s@mail.com	9aF2...	2026-06-10 09:15	Parramatta NSW	L1a4...	2026-06-10 09:15	PLACED			
9aF2...	2026-06-11 08:05	priya@newmail.com				L1a4...	2026-06-11 16:20	SHIPPED			

① POINT-IN-TIME — one row per key per snapshot; columns hold the chosen LOAD_DTS per satellite

PIT_CUSTOMER (daily snapshots of CUST-501)			
SNAPSHOT_DTS (PK)	CUSTOMER_HK (PK,FK)	SAT_PROFILE_LOAD_DTS	SAT_ADDRESS_LOAD_DTS
2026-06-10 23:59	9aF2...	06-10 09:15	06-10 09:15
2026-06-11 23:59	9aF2...	06-11 08:05	06-10 09:15

HOW TO READ THE 06-11 ROW

SAT_ADDRESS_LOAD_DTS = 06-10 09:15 comes straight from the one address row above — it never advanced because the address satellite has only one row.

The profile date advanced to 08:05 because the email changed (event T3).

PIT_ORDER_ITEM (daily snapshots of line L1a4...)			
SNAPSHOT_DTS (PK)	ORDER_ITEM_HK (PK,FK)	SAT_ITEM_LOAD_DTS	SAT_STATUS_LOAD_DTS
2026-06-10 23:59	L1a4...	06-10 09:15	06-10 09:15
2026-06-11 23:59	L1a4...	06-10 09:15	06-11 16:20

PIT_CUSTOMER: profile → 08:05 (new email); address → 06-10 09:15 (unchanged). PIT_ORDER_ITEM: item details unchanged (09:15); status = 16:20 (SHIPPED).

A query joins each satellite with TWO equals: ON sat.hk = pit.hk AND sat.load_dts = pit.sat_x.load_dts No BETWEEN, no MAX(), no correlated subquery — just equi-joins an optimiser loves.

WHY IT'S CHEAPER — the same “as-of 11 June” question, without a PIT and with one

WITHOUT PIT — date-range hunt in every satellite		WITH PIT — equi-join on the pre-picked dates	
<pre>SELECT p.email_address, s.order_status FROM hub_customer h ... JOIN sat_customer_profile p ON p.customer_hk = h.customer_hk AND p.load_dts = (SELECT MAX(load_dts) FROM sat_customer_profile WHERE customer_hk = h.customer_hk AND load_dts <= '2026-06-11 23:59') -- One correlated MAX() per satellite — scans grow with history.</pre>		<pre>SELECT p.email_address, s.order_status FROM pit_customer pit JOIN sat_customer_profile p ON p.customer_hk = pit.customer_hk AND p.load_dts = pit.sat_profile_load_dts JOIN pit_order_item pio ON pio.snapshot_dts = pit.snapshot_dts JOIN sat_order_status s ON s.order_item_hk = pio.order_item_hk AND s.load_dts = pio.sat_status_load_dts WHERE pit.snapshot_dts = '2026-06-11 23:59'; -- pure equi-joins</pre>	

② BRIDGE — pre-joins the Hub & Link keys for a whole subject area into one snapshot row

BRDG_ORDER_SALES (2026-06-11 snapshot — both order lines from Page 6, keys pre-joined)						
SNAPSHOT_DTS (PK)	ORDER_ITEM_HK (PK,FK)	CUSTOMER_HK	ORDER_HK	PRODUCT_HK	CHANNEL_HK	PAYMENT_METHOD_HK
2026-06-11 23:59	L1a4...	9aF2...	7e1D...	5fB3...	6cD1...	7aB5...
2026-06-11 23:59	L9c2...	4bC7...	2cA9...	8dE6...	3eF8...	1dC4...

THE BRIDGE IS A FACT-TABLE SKELETON

One row already knows: which customer, order, product, channel and payment method belong to each order line — no Hub→Link→Hub walking. Join it to PIT-resolved satellites for measures → VF_SALES (Page 7). Drop & rebuild nightly; the Raw Vault is untouched (Page 3).

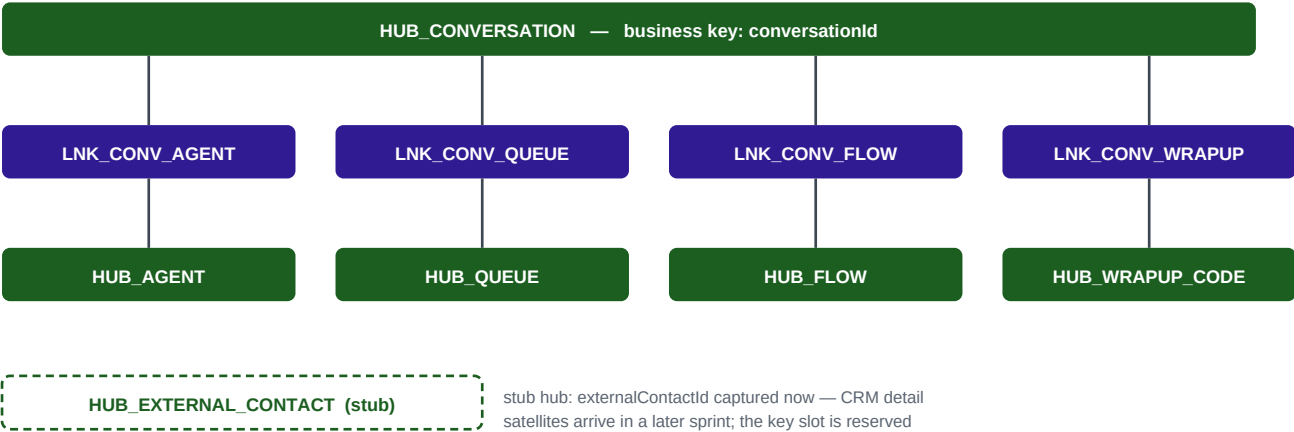
PIT + BRIDGE TOGETHER = THE STAR ON PAGE 7

The Bridge supplies the dimension keys (one row per order line); the PIT supplies the correct satellite version dates for each measure and attribute. Join them and you get exactly VF_SALES + the VDIM_views on Page 7 — built by the Page 8 SQL, rebuilt from the vault any time, with zero date-range logic at query time.

Two real conversation-detail payloads — a voice call (IVR → queue → agent) and an async web message (bot → workflow) — classified into Hubs, Links & Satellites

Source: /api/v2/analytics/conversations/details

TARGET MODEL AT A GLANCE — HUB_CONVERSATION anchors everything the payload contains



SATELLITES IN THIS MODEL

- On HUB_CONVERSATION:
- SAT_CONVERSATION_DETAILS — start/end, direction, min MOS (versions over time)
 - SAT_SESSION — multi-active: one row per participant-session (SSQN), no own key
 - SAT_SEGMENT — multi-active at the finest grain: participant · session · segmentStart
 - SAT_CONV_EFFECTIVITY — open / closed (P.12)
- On the Links:
- SAT_AGENT_METRICS (LNK_CONV_AGENT)
 - SAT_FLOW_EXECUTION (LNK_CONV_FLOW)
 - SAT_WRAPUP_NOTE (LNK_CONV_WRAPUP)
- Multi-active satellite = many simultaneously active child rows; PK adds a sub-sequence.

ATTRIBUTE-BY-ATTRIBUTE — where each JSON element lands, and why

GENESYS conversation-detail JSON → DATA VAULT		
GENESYS ATTRIBUTE (JSON path)	LANDS IN	WHY — DV 2.0 CLASSIFICATION
conversationId	HUB_CONVERSATION	The business key of an interaction — unique, enterprise-wide, never reused
conversationStart / End · originatingDirection · min MOS	SAT_CONVERSATION_DETAILS	Descriptive context that versions over time — End arrives DAYS later for async (Page 12)
participants[].purpose + sessions[] (mediaType, ani, dnis, direction)	SAT_SESSION (multi-active)	5 participants live on one call, none with a standalone key → sub-sequence = participantId-sessionId
sessions[].segments[] (type, start/end, disconnectType, sipCodes)	SAT_SEGMENT (multi-active)	Finest grain; sub-sequence = participantId-sessionId-segmentStart — the natural dedupe key on re-delivery
selectedAgentId / agent userId	HUB_AGENT + LNK_CONV_AGENT	Who handled it = a relationship between two business keys
metrics: tAlert, tAnswered, tTalk, tAcw, tHandle, nOverSla	SAT_AGENT_METRICS (on the Link)	Measures describe the conversation↔agent relationship — not either key alone
queueId (acd participant / surveys[])	HUB_QUEUE + LNK_CONV_QUEUE	Routing relationship; the queue is a reusable enterprise key
flow.flowId (IVR, in-queue, bot, workflow)	HUB_FLOW + LNK_CONV_FLOW	Each flow is a key; one execution inside this conversation = one Link row
flow detail: exitReason, flowVersion, tFlow, recognitionFailureReason, outcomes[]	SAT_FLOW_EXECUTION (on the Link)	Describes that execution of that flow — versioned like any Satellite
segments[].wrapUpCode / wrapUpNote	HUB_WRAPUP_CODE + LNK_CONV_WRAPUP; note → SAT_WRAPUP_NOTE	Code = business key; free-text note = descriptive — PII scrubbed in STAGING (hard rule, Page 4)
externalContactId	HUB_EXTERNAL_CONTACT (stub hub)	Key captured now; CRM detail satellites are out of scope this sprint — filled in later
proposedAgents[] · mediaEndpointStats[]	SAT_ROUTING_PROPOSAL / SAT_MEDIA_QUALITY (multi-active)	Split by TYPE of data & rate of change — noisy telemetry kept away from stable context

Populated rows from the two payloads — GUIDs truncated, phone numbers faked, wrap-up note paraphrased: PII never enters the Raw Vault

Voice call 2b13f490... + async web message 361f7e4e...

1 HUBS — one row per business key seen across BOTH payloads (REC_SRC = GEN_API)

HUB_CONVERSATION			
CONVERSATION_HK (PK)	CONVERSATION_ID	LOAD_DTS	REC_SRC
Cva7...	2b13f490...	04-16 03:00	GEN_API
Cm3e...	361f7e4e...	03-01 14:00	GEN_API

HUB_AGENT			
AGENT_HK (PK)	USER_ID	LOAD_DTS	REC_SRC
Ag5d...	db6884ce...	04-16 03:00	GEN_API

HUB_QUEUE			
QUEUE_HK (PK)	QUEUE_ID	LOAD_DTS	REC_SRC
Qu91...	01231be0...	04-16 03:00	GEN_API

HUB_FLOW			
FLOW_HK (PK)	FLOW_ID	LOAD_DTS	REC_SRC
FI1a...	e1d2b9d6...	04-16 03:00	GEN_API
FI2b...	6b69b2ba...	04-16 03:00	GEN_API
FI3c...	63c7a58d...	04-16 03:00	GEN_API
FI4d...	76833c6b...	03-01 14:00	GEN_API
FI5e...	32e46c26...	03-01 14:00	GEN_API

HUB_WRAPUP_CODE			
WRAPUP_HK (PK)	WRAPUP_CODE_ID	LOAD_DTS	REC_SRC
Wc76...	766891af...	04-16 03:00	GEN_API

HUB_EXTERNAL_CONTACT (stub — key only)			
CONTACT_HK (PK)	EXTERNAL_CONTACT_ID	LOAD_DTS	REC_SRC
Xc22...	a22ff817...	04-16 03:00	GEN_API
Xc55...	55f90237...	03-01 14:00	GEN_API

2 LINKS — one row per unique key combination (LNK_CONV_QUEUE follows the same pattern, omitted for space)

LNK_CONV_AGENT				
CONV_AGENT_HK (PK)	CONVERSATION_HK	AGENT_HK	LOAD_DTS	REC_SRC
CA19...	Cva7...	Ag5d...	04-16 03:00	GEN_API

LNK_CONV_FLOW				
CONV_FLOW_HK (PK)	CONVERSATION_HK	FLOW_HK	LOAD_DTS	REC_SRC
CF01...	Cva7...	FI1a...	04-16 03:00	GEN_API
CF02...	Cva7...	FI2b...	04-16 03:00	GEN_API
CF03...	Cva7...	FI3c...	04-16 03:00	GEN_API
CF04...	Cm3e...	FI4d...	03-01 14:00	GEN_API
CF05...	Cm3e...	FI5e...	03-01 14:00	GEN_API

LNK_CONV_WRAPUP		
CONV_WRAPUP_HK (PK)	CONVERSATION_HK	WRAPUP_HK
CW44...	Cva7...	Wc76...

3 SATELLITES — multi-active SAT_SESSION (sub-sequence = participant-session) and the Link satellites

SAT_SESSION (parent: HUB_CONVERSATION — multi-active)								
CONV_HK (PK,FK)	LOAD_DTS (PK)	SSQN (PK)	HASH_DIFF	REC_SRC	PURPOSE	MEDIA	ADDRESS (faked)	DIRECTION
Cva7...	04-16 03:00	cust-58b2...	s1a2...	GEN_API	customer	voice	+61 400 000 123	inbound
Cva7...	04-16 03:00	ivr-0006...	s2b3...	GEN_API	ivr	voice	+61 2 9000 0000	inbound
Cva7...	04-16 03:00	acd-c899...	s3c4...	GEN_API	acd	voice	+61 2 9000 0000	inbound
Cva7...	04-16 03:00	agent-3432...	s4d5...	GEN_API	agent	voice	+61 2 9000 0000	inbound
Cm3e...	03-01 14:00	cust-1338...	s5e6...	GEN_API	customer	message	web · 4ee0...	inbound
Cm3e...	03-01 14:00	bot-2970...	s6f7...	GEN_API	botflow	message	web msg	inbound

SAT_AGENT_METRICS (parent: LNK_CONV_AGENT)								
CONV_AGENT_HK	LOAD_DTS	HASH_DIFF	REC_SRC	tALERT_s	tTALK_s	tACW_s	tHANDLE_s	nOverSLA
CA19...	04-16 03:00	m7f1...	GEN_API	16.6	706.7	172.6	879.3	1

SAT_WRAPUP_NOTE (parent: LNK_CONV_WRAPUP — note paraphrased, PII removed)				
CONV_WRAPUP_HK	LOAD_DTS	HASH_DIFF	REC_SRC	NOTE (scrubbed in staging)
CW44...	04-16 03:00	w3c9...	GEN_API	Offer enquiry; structure explained; starting part-time.

SAT_FLOW_EXECUTION (parent: LNK_CONV_FLOW)								
CONV_FLOW_HK	LOAD_DTS	HASH_DIFF	REC_SRC	FLOW_NAME	FLOW_TYPE	EXIT_REASON	tFLOW_s	VER
CF01...	04-16 03:00	fa01...	GEN_API	Main Contact IVR	INBOUNDCALL	TRANSFER	32.4	8
CF02...	04-16 03:00	fb02...	GEN_API	Switch Open Hours	INBOUNDCALL	TRANSFER	10.6	7
CF03...	04-16 03:00	fc03...	GEN_API	In-Queue Flow	INQUEUECALL	—	—	3
CF04...	03-01 14:00	fd04...	GEN_API	Web Messaging Entry	SHORTMSG	DISCONNECT	302.4	31
CF05...	03-01 14:00	fe05...	GEN_API	Welcome Chatbot	DIGITALBOT	RECOG_FAIL...	301.5	127

PII & ANONYMISATION — A HARD RULE IN STAGING (Page 4)

Phone numbers replaced with fakes (+61 400 000 123), GUIDs truncated, the wrap-up free text paraphrased and flow / queue names genericised BEFORE the vault load. Scrubbing is a HARD rule: it standardises without changing business meaning — so the vault stays both auditable and privacy-safe.

SAME LOADER, NEW SOURCE

Every table above is loaded by exactly the Page-8 SQL patterns — hubs and links are no-ops on re-delivery, satellites insert only on HASH_DIFF change. The one twist the contact-centre source adds is the asynchronous message conversation that stays OPEN for up to 3 days — Page 12 shows how the vault absorbs that.

A web-message conversation stays active until 72 h of inactivity; conversationEnd is NULL until then — the vault absorbs every re-extract as new rows, never updates

Illustrative timeline; the sample payload closed same-day

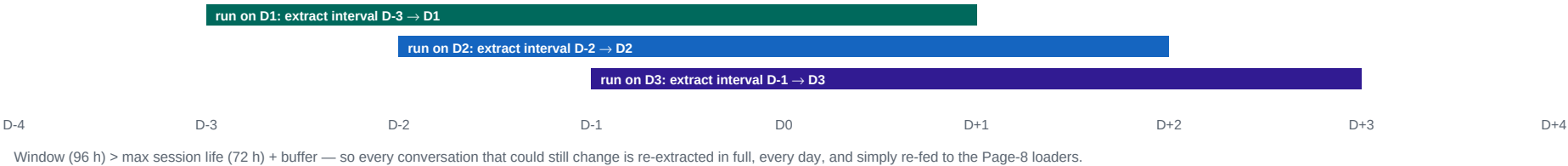
WHAT MAKES MESSAGING DIFFERENT FROM VOICE

A voice call ends when someone hangs up — one extract captures it whole. Async web messaging keeps the conversation OPEN: the customer can leave and reply hours or days later. Until 72 h pass with no activity, conversationEnd is NULL and the SAME conversationId keeps returning from the API with changed content.

TIMELINE — one conversation across four days (conversation 361f7e4e...)



ROLLING LOOKBACK EXTRACT — every daily run re-queries a window wider than the 72 h session limit



WHAT LANDS IN THE VAULT — the open → closed transition is just more INSERT-ONLY rows

SAT_CONVERSATION_DETAILS (parent: HUB_CONVERSATION)

CONV_HK (PK,FK)	LOAD_DTS (PK)	HASH_DIFF	REC_SRC	CONVERSATION_END	oMSG_COUNT	STATE
Cm3e...	03-01 14:00	h1a2...	GEN_API	NULL	2	OPEN
Cm3e...	03-02 14:00	h9f4...	GEN_API	NULL	5	OPEN
Cm3e...	03-04 14:00	hc77...	GEN_API	03-04 13:22	5	CLOSED

SAT_CONV_EFFECTIVITY (parent: HUB_CONVERSATION — no HASH_DIFF, per Page 2)

CONV_HK (PK,FK)	LOAD_DTS (PK)	REC_SRC	EFFECTIVE_FROM	EFFECTIVE_TO	IS_OPEN
Cm3e...	03-01 14:00	GEN_API	03-01 13:17	9999-12-31	Y
Cm3e...	03-04 14:00	GEN_API	03-01 13:17	03-04 13:22	N

NO UPDATES

open → closed
= one more
version row.
Old rows stay
untouched.

The D+2 run (03-03 14:00) re-extracted the conversation too — identical payload → identical HASH_DIFF (h9f4...) → 0 rows inserted. Idempotent by construction.

THE LOADER PATTERN — how data engineers run this in production

1. Each run calls conversation_detail_query for the interval [now – 96 h → now] and lands the raw JSON in staging — hard rules flatten it, scrub PII, compute hash keys.

2. Hubs & Links: re-delivered conversations are no-ops (NOT EXISTS on the hash key — Page 8 panels 1–2). New flows / agents / queues insert exactly once.

3. Satellites: re-delivered content inserts ONLY when HASH_DIFF changes (Page 8 panel 3). Multi-active SAT_SEGMENT keys on participant-session-segmentStart, so re-delivered segments dedupe naturally — the same idea as a MERGE keyed at segment grain, achieved with pure inserts.

4. When conversationEnd finally arrives (≤ 3 days late), it is just another satellite version + an effectivity close — marts pick the LATEST row (or the PIT, Page 3), so an open conversation shows NULL end until the closing version supersedes it. No reload, no update, no special late-arrival code path.

WHY THIS FITS DATA VAULT SO WELL

Late-arriving, mutating, re-delivered source data is the exact failure mode insert-only architecture was designed for: the 3-day window just means a conversation grows extra satellite versions for a few days before its final state settles — and every intermediate state remains queryable forever (Page 2: what was true, when).

THE QUESTION

“How is a Satellite different from a Link? And how is a Link-Satellite different from a Bridge that records all the Hub keys?” All three touch the order line — but a Link records THAT keys relate, a Link-Satellite records WHAT was true about that relationship over time, and a Bridge pre-joins MANY keys into one disposable snapshot so analysts never walk Hub→Link→Hub. Link & Satellite are Raw-Vault truth; the Bridge is a rebuildable Information-Mart shortcut.

THE SAME ORDER LINE, THREE WAYS — order line L1a4... (ORD-10001 · PRD-2001 · CUST-501) as it appears in each structure

LNK_ORDER_ITEM				SAT_ORDER_STATUS			BRDG_ORDER_SALES						
LINK — the relationship spine (Raw Vault)				LINK-SATELLITE — attributes + history (Raw Vault)			BRIDGE — many keys pre-joined (Information Mart)						
ORDER_ITEM_HK	ORDER_HK	PRODUCT_HK	CUSTOMER_HK	ORDER_ITEM_HK	LOAD_DTS	ORDER_STATUS	SNAPSHOT_DTS	ORDER_ITEM_HK	CUSTOMER_HK	ORDER_HK	PRODUCT_HK	CHANNEL_HK	PAYMENT_METHOD_HK
L1a4...	7e1D...	5fB3...	9aF2...	L1a4...	06-10 09:15	PLACED	2026-06-11 23:59	L1a4...	9aF2...	7e1D...	5fB3...	6cD1...	7aB5...
Keys only — “did these relate?” No amounts, no dates, no status. Insert-once; never updated.				L1a4... 06-11 16:20 SHIPPED Attributes — “what was true, when?” Two rows = two versions over time. One parent key; carries the payload.			ALL keys — “which relate, fast?” No measures — a fact-table skeleton. Disposable; rebuilt from the vault anytime.						

WHY THEY DIFFER — two comparisons that trip people up

LINK vs LINK-SATELLITE	LINK-SATELLITE vs BRIDGE
LNK_ORDER_ITEM records only THAT the order line happened — order + product + customer hash keys plus audit columns, nothing else. SAT_ORDER_ITEM_DETAILS / SAT_ORDER_STATUS hang off that link and carry the payload: quantity, price, line total — and status over time (PLACED → SHIPPED as two insert-only rows). Link = grain & integration (“did they relate?”). Link-Satellite = content & history (“what was true, when?”).	A Link-Satellite is Raw-Vault truth: permanent, insert-only, ONE parent, storing descriptive attributes and versioning them over time. BRDG_ORDER_SALES is an Information-Mart shortcut: derived, disposable, rebuilt nightly. It pre-joins MANY keys (customer, order, product, channel, payment) into one snapshot row — no Hub→Link→Hub walk. Content vs navigation: the Satellite stores attributes + history about ONE relationship; the Bridge stores keys only, pre-joined across many.

SIDE BY SIDE — the full comparison

	LINK LNK_ORDER_ITEM	LINK-SATELLITE SAT_ORDER_...	BRIDGE BRDG_ORDER_SALES
Layer	Raw Vault	Raw Vault	Information Mart
What it holds	Hub keys of ONE relationship	Descriptive attributes + history	MANY hub & link keys, pre-joined
Grain	One order line (unit of work)	parent_HK + LOAD_DTS	One key-combo per SNAPSHOT_DTS
Keeps history?	Insert-once — never changes	Versioned — new row per change	Snapshot only — no attribute history
Permanent?	Yes — source of truth	Yes — source of truth	No — disposable, rebuilt anytime
Keys vs measures	Keys only, no measures	Measures + 1 parent key, few FKs	All FKs, no measures
Purpose	The relationship spine	Content + time-variance	Query speed / join elimination
Answers	“did these keys relate?”	“what was true, and when?”	“which keys relate — made fast”

ONE-LINE MNEMONIC

The Link is the relationship; the Link-Satellite is everything you’d say about that relationship over time; the Bridge is a flattened map of keys you build later so you never walk Hub→Link→Hub at query time.

Mirror images: a Satellite = attributes + ONE parent key (carries CONTENT); a Bridge = ALL keys + NO attributes (carries NAVIGATION). Same rows as Pages 1–3 & the Page 9 worked example.